

Jour 3 : Pipelines Avances - Multi-Projets et Pipelines Dynamiques

Programme de la journee

Horaire	Sujet
09h00 - 10h30	Pipelines multi-projets et includes
10h30 - 10h45	Pause
10h45 - 12h15	Pipelines dynamiques (child pipelines)
12h15 - 13h30	Pause dejeuner
13h30 - 15h00	Workflow rules avancees
15h00 - 15h15	Pause
15h15 - 16h45	Efficacite des pipelines (DAG, retry, timeout)
16h45 - 17h00	Recapitulatif et Q&A

Partie 1 : Pipelines multi-projets et includes

L'analogie de la bibliotheque

Imaginez que votre fichier `.gitlab-ci.yml` est une **recette de cuisine**. Au debut, vous ecrivez toute la recette dans un seul cahier. Mais quand vous avez 50 projets, vous recopiez les memes instructions partout : installer les dependances, lancer les tests, deployer...

C'est comme si 50 cuisiniers recopiaient a la main la meme recette de sauce tomate.

Si la recette change, il faut modifier **50 cahiers** !

La solution ? Creer un **livre de recettes central** et dire a chaque cuisinier :

"Va lire la page 12 du livre pour la sauce tomate." C'est exactement ce que fait `include`.

```
SANS include (duplication) :
Projet A : .gitlab-ci.yml (200 lignes, dont 150 copiees)
Projet B : .gitlab-ci.yml (200 lignes, dont 150 copiees)
Projet C : .gitlab-ci.yml (200 lignes, dont 150 copiees)

AVEC include (centralisation) :
Projet templates : templates/test.yml, templates/deploy.yml
Projet A : .gitlab-ci.yml (50 lignes + include)
Projet B : .gitlab-ci.yml (50 lignes + include)
Projet C : .gitlab-ci.yml (50 lignes + include)
```

Les 4 types d'include

GitLab propose **4 facons** d'inclure des fichiers de configuration externes.

Chacune correspond a un cas d'usage different.

Type	Source	Cas d'usage
<code>include:local</code>	Meme projet, meme branche	Decouper un gros fichier
<code>include:file</code>	Autre projet GitLab	Templates partages entre projets
<code>include:template</code>	Templates officiels GitLab	Bonnes pratiques pre-faites
<code>include:remote</code>	URL externe (HTTP/HTTPS)	Templates open-source

include:local - Decouper son propre projet

`include:local` inclut un fichier **du meme projet**, sur la **meme branche**.

C'est comme si vous disiez : "Va lire le chapitre 3 de CE cahier."

```
# .gitlab-ci.yml (fichier principal)

# On inclut des fichiers qui sont dans LE MEME projet
include:
  - local: '/ci/tests.yml'      # Fichier de tests
  - local: '/ci/deploy.yml'    # Fichier de deploiement
  - local: '/ci/quality.yml'   # Fichier de qualite de code

# On peut ajouter des jobs supplementaires ici
stages:
  - test
  - quality
  - deploy
```

```
# ci/tests.yml (fichier inclus)

# Ce fichier ne contient QUE les jobs de test
# Il sera "fusionne" avec le fichier principal

unit_tests:
  stage: test
  image: node:18-alpine
  script:
    - npm ci
    - npm test

integration_tests:
  stage: test
  image: node:18-alpine
  script:
    - npm ci
    - npm run test:integration
```

Quand utiliser `include:local` ?

- Votre `.gitlab-ci.yml` depasse 200 lignes
- Vous voulez organiser par theme (test, deploy, quality)
- Vous voulez que chaque equipe gere sa partie du pipeline

include:file - Partager entre projets

`include:file` inclut un fichier **d'un autre projet GitLab**. C'est la cle des pipelines multi-projets.

C'est comme si vous disiez : "Va lire la page 12 du livre de recettes"

qui se trouve dans la bibliotheque centrale."

```
# .gitlab-ci.yml du Projet A
# On inclut un fichier qui vient du Projet "ci-templates"

include:
  - project: 'mon-groupe/ci-templates' # Chemin du projet source
    ref: main                          # Branche a utiliser
    file: '/templates/nodejs-test.yml'  # Chemin du fichier dans ce projet

  - project: 'mon-groupe/ci-templates'
    ref: v2.1.0                        # On peut utiliser un tag !
    file: '/templates/deploy-production.yml'

stages:
  - test
  - deploy
```

```
# Projet "ci-templates" > templates/nodejs-test.yml
# Ce fichier est reutilise par 50 projets !

.test_template:
  image: node:18-alpine
  before_script:
    - npm ci
  cache:
    key: ${CI_COMMIT_REF_SLUG}
    paths:
      - node_modules/

test_unitaires:
  extends: .test_template
  stage: test
  script:
    - npm test
  coverage: '/Lines\s*:\s*(\d+\.\d*)%/'
```

Bonne pratique : versionner avec des tags

```
# MAUVAIS : on pointe sur "main" qui change tout le temps
include:
  - project: 'mon-groupe/ci-templates'
    ref: main
    file: '/templates/deploy.yml'

# BON : on pointe sur un tag stable
include:
  - project: 'mon-groupe/ci-templates'
    ref: v2.1.0
    file: '/templates/deploy.yml'

# ENCORE MIEUX : on peut facilement upgrader
# En changeant juste le numero de version
```

include:template - Templates officiels GitLab

GitLab fournit des templates **pre-configures** pour les cas courants.

C'est comme si le fabricant de votre four vous donnait un livre de recettes
deja pret avec les meilleurs reglages.

```
# Utiliser les templates officiels de GitLab
include:
  # Template pour l'analyse de securite SAST
  - template: Security/SAST.gitlab-ci.yml

  # Template pour la detection de secrets
  - template: Security/Secret-Detection.gitlab-ci.yml

  # Template pour les tests de qualite de code
  - template: Code-Quality.gitlab-ci.yml

  # Template pour les pages GitLab
  - template: Pages/HTML.gitlab-ci.yml
```

Où trouver la liste des templates ?

- Dans GitLab : Projet > CI/CD > Editor > Browse templates
- Sur le depot :
<https://gitlab.com/gitlab-org/gitlab/-/tree/master/lib/gitlab/ci/templates>

include:remote - Fichiers externes via URL

`include:remote` charge un fichier depuis **n'importe quelle URL HTTP/HTTPS**.

C'est comme commander un livre de recettes sur Internet.

```
# Inclure un fichier depuis une URL externe
include:
  - remote: 'https://example.com/ci/templates/nodejs.yml'

# Depuis un depot GitHub par exemple
  - remote: 'https://raw.githubusercontent.com/mon-org/ci-templates/main/deploy.yml'
```

Attention : Le fichier doit être accessible publiquement (pas d'authentification).

Pour des fichiers privés, utilisez `include:file` avec un projet GitLab.

Combiner plusieurs types d'include

```
# On peut melanger tous les types dans un seul fichier !
include:
  # Depuis le meme projet
  - local: '/ci/variables.yml'

  # Depuis un autre projet GitLab
  - project: 'devops/templates'
    ref: v3.0.0
    file: '/templates/docker-build.yml'

  # Depuis les templates officiels
  - template: Security/SAST.gitlab-ci.yml

  # Depuis une URL externe
  - remote: 'https://example.com/ci/lint.yml'

stages:
  - build
  - test
  - security
  - deploy
```

Declenchement de pipelines cross-projet avec trigger

L'instruction trigger permet de **declencher le pipeline d'un autre projet**.

C'est comme si le chef cuisinier du restaurant A appelait le chef du restaurant B pour lui dire : "Prepare ta specialite, j'en ai besoin pour mon menu !"

```
# .gitlab-ci.yml du Projet A (le declencheur)

stages:
  - build
  - test
  - trigger

build:
  stage: build
  script:
    - echo "Construction du projet A"

test:
  stage: test
  script:
    - echo "Tests du projet A"

# Ce job declenche le pipeline du Projet B
declencher_projet_b:
  stage: trigger
  trigger:
    project: mon-groupe/projet-b    # Chemin du projet a declencher
    branch: main                    # Branche sur laquelle declencher
    strategy: depend                 # Attendre le resultat du pipeline declenche
```

Le mot-cle strategy: depend

```
# SANS strategy: depend
# Le job "declencher_projet_b" passe en succes immediatement
# apres avoir declenche le pipeline. Il ne verifie PAS le resultat.

declencher_sans_attendre:
  stage: trigger
  trigger:
    project: mon-groupe/projet-b

# AVEC strategy: depend
# Le job attend que le pipeline du Projet B se termine
# Si le Projet B echoue, ce job echoue aussi !

declencher_avec_attente:
  stage: trigger
  trigger:
    project: mon-groupe/projet-b
    strategy: depend
```

Passer des variables entre pipelines

Quand vous declenchez un pipeline downstream, vous pouvez lui **passer des variables**.

C'est comme donner une note au chef du restaurant B :

"Voici la commande du client : 2 plats, sans gluten."

```
# Projet A declenche Projet B en lui passant des variables
declencher_deploiement:
  stage: trigger
  variables:
    # Ces variables seront disponibles dans le pipeline du Projet B
    VERSION: $CI_COMMIT_TAG
    ENVIRONNEMENT: 'production'
    IMAGE_TAG: '$CI_REGISTRY_IMAGE:$CI_COMMIT_SHORT_SHA'
  trigger:
    project: mon-groupe/projet-deploiement
    branch: main
    strategy: depend
```

```
# .gitlab-ci.yml du Projet B (le projet declenche)
# Les variables passees par le Projet A sont disponibles ici

deploy:
  stage: deploy
  script:
    - echo "Deploiement de la version $VERSION"
    - echo "Environnement cible $ENVIRONNEMENT"
    - echo "Image Docker $IMAGE_TAG"
    - kubectl set image deployment/app app=$IMAGE_TAG
```

Schema recapitulatif des pipelines multi-projets

```

Projet A (pipeline upstream)
|
|-- build --> test --> trigger
|
|   |
|   | declenche (avec variables)
|   v
|   Projet B (pipeline downstream)
|   |
|   |-- deploy --> verify
|   |
|   | resultat (si strategy: depend)
|   v
|   Projet A recoit succes/echec

```

Partie 2 : Pipelines dynamiques (child pipelines)

L'analogie du menu du jour

Imaginez un restaurant qui change son menu **chaque jour** en fonction des ingrédients disponibles. Le chef ne peut pas imprimer un menu fixe à l'avance. Il doit :

1. Regarder les ingrédients disponibles ce matin (analyser le contexte)
2. Ecrire le menu du jour (générer la configuration)
3. Donner le menu aux cuisiniers (déclencher le pipeline)

C'est exactement le pattern **generate + trigger** des child pipelines.

La différence entre trigger cross-projet et child pipeline

Aspect	Cross-projet (trigger)	Child pipeline
Où	Déclenche un autre projet	Déclenche dans le même projet
Fichier	Le .gitlab-ci.yml de l'autre	Un fichier génère dynamiquement
Visibilité	Pipeline séparé	Imbriqué dans le pipeline parent
Variables	Passage explicite	Héritage automatique
Cas d'usage	Microservices, monorepos	Pipelines adaptatifs

Le pattern generate + trigger

Le principe est simple en 2 étapes :

1. Un job **génère** un fichier YAML (la configuration du child pipeline)
2. Un autre job **déclenche** ce fichier comme un sous-pipeline

```

# .gitlab-ci.yml

stages:
  - generate
  - trigger-child
  - deploy

# Etape 1 : Generer dynamiquement la configuration du child pipeline
generer_config:
  stage: generate
  image: alpine:latest
  script:
    # On cree un fichier YAML a la volee !
    - |
      cat > child-pipeline.yml << 'HEREDOC'
      stages:
        - test
        - build

      test_dynamique:
        stage: test
        image: node:18-alpine
        script:
          - echo "Test genere dynamiquement"
          - npm ci
          - npm test

      build_dynamique:
        stage: build
        image: docker:24
        script:
          - echo "Build genere dynamiquement"
          - docker build -t mon-app .
      HEREDOC
  artifacts:
    paths:
      - child-pipeline.yml # IMPORTANT : le fichier doit etre un artifact

# Etape 2 : Declencher le child pipeline avec le fichier genere
lancer_child:
  stage: trigger-child
  trigger:
    include:
      - artifact: child-pipeline.yml # On utilise l'artifact genere
        job: generer_config          # Le job qui a cree l'artifact
  strategy: depend

```

Pourquoi generer dynamiquement ?

La puissance des child pipelines, c'est de pouvoir **adapter le pipeline au contexte**.

Exemple concret : un monorepo

Vous avez un seul depot Git avec 5 services :

```

monorepo/
  ■■■ service-api/
  ■■■ service-web/
  ■■■ service-worker/
  ■■■ service-auth/
  ■■■ service-notifications/

```

Si seul service-api/ a change, pourquoi tester les 5 services ?

```

# .gitlab-ci.yml du monorepo

stages:
  - detect
  - trigger-child

# Etape 1 : Detecter quels services ont change
detecter_changements:
  stage: detect
  image: alpine:latest
  script:
    # On commence par installer git
    - apk add --no-cache git

    # On detecte quels dossiers ont ete modifies
    - CHANGED=$(git diff --name-only HEAD~1 HEAD)
    - echo "Fichiers modifies $CHANGED"

    # On genere un pipeline UNIQUEMENT pour les services modifies
    - echo 'stages:' > child-pipeline.yml
    - echo '  - test' >> child-pipeline.yml
    - echo '  - build' >> child-pipeline.yml

    # Si service-api a change, on ajoute son test
    - |
      if echo "$CHANGED" | grep -q "^service-api/"; then
        cat >> child-pipeline.yml << 'HEREDOC'

        test_api:
          stage: test
          image: node:18-alpine
          script:
            - cd service-api
            - npm ci
            - npm test

        build_api:
          stage: build
          script:
            - cd service-api
            - docker build -t service-api .
        HEREDOC
      fi

    # Si service-web a change, on ajoute son test
    - |
      if echo "$CHANGED" | grep -q "^service-web/"; then
        cat >> child-pipeline.yml << 'HEREDOC'

        test_web:
          stage: test
          image: node:18-alpine
          script:
            - cd service-web
            - npm ci
            - npm test

        build_web:

```

Generation avancee avec un script Python

Pour des cas plus complexes, on peut utiliser un vrai langage de programmation pour generer le YAML.

```
# .gitlab-ci.yml

stages:
  - generate
  - child

generer_avec_python:
  stage: generate
  image: python:3.11-alpine
  script:
    - pip install pyyaml
    - python generate_pipeline.py
  artifacts:
    paths:
      - generated-pipeline.yml
```

```
# generate_pipeline.py
import yaml
import os
import subprocess

# Detecter les fichiers modifies
result = subprocess.run(
    ['git', 'diff', '--name-only', 'HEAD~1', 'HEAD'],
    capture_output=True, text=True
)
changed_files = result.stdout.strip().split('\n')

# Construire dynamiquement le pipeline
pipeline = {
    'stages': ['test', 'build'],
}

# Pour chaque service modifie, ajouter des jobs
services = ['service-api', 'service-web', 'service-worker']
for service in services:
    if any(f.startswith(f'{service}/') for f in changed_files):
        # Ajouter un job de test
        pipeline[f'test_{service}'] = {
            'stage': 'test',
            'image': 'node:18-alpine',
            'script': [
                f'cd {service}',
                'npm ci',
                'npm test',
            ],
        }
        # Ajouter un job de build
        pipeline[f'build_{service}'] = {
            'stage': 'build',
            'image': 'docker:24',
            'script': [
                f'cd {service}',
                f'docker build -t {service} .',
            ],
        }

# Ecrire le fichier YAML
with open('generated-pipeline.yml', 'w') as f:
    yaml.dump(pipeline, f, default_flow_style=False)

print("Pipeline genere avec succes :")
with open('generated-pipeline.yml') as f:
    print(f.read())
```

Rules avec variables dans les child pipelines

On peut utiliser rules avec des variables pour contrôler finement quels jobs s'exécutent dans le child pipeline.

```
# child-pipeline.yml avec rules et variables

variables:
  DEPLOY_ENV: 'staging'

stages:
  - test
  - deploy

test:
  stage: test
  script:
    - npm test
  rules:
    # Toujours exécuter les tests
    - when: always

deploy_staging:
  stage: deploy
  script:
    - echo "Déploiement sur staging"
  rules:
    # Seulement si DEPLOY_ENV vaut "staging"
    - if: '$DEPLOY_ENV == "staging"'

deploy_production:
  stage: deploy
  script:
    - echo "Déploiement en production"
  rules:
    # Seulement si DEPLOY_ENV vaut "production"
    - if: '$DEPLOY_ENV == "production"'
    # ET seulement sur la branche main
    - if: '$CI_COMMIT_BRANCH == "main"'
```

```
# Le parent peut passer des variables au child pipeline
lancer_child:
  stage: trigger-child
  variables:
    DEPLOY_ENV: 'production' # Cette variable sera utilisée par le child
  trigger:
    include:
      - artifact: child-pipeline.yml
        job: generer_config
  strategy: depend
```

Partie 3 : Workflow rules avancées

L'analogie du gardien de porte

Imaginez votre pipeline comme un **concert**. Avant que le concert commence, il y a un gardien à l'entrée qui décide si le concert a lieu ou non :

- "Il pleut ? Pas de concert en plein air." (condition meteo)
- "Moins de 10 billets vendus ? On annule." (condition de rentabilite)
- "C'est un jour ferie ? Report." (condition de calendrier)

Le mot-cle workflow:rules est ce **gardien**. Il decide si **tout le pipeline** se lance ou non, AVANT meme que les jobs individuels ne soient evalues.

workflow:rules - Controler le lancement du pipeline

```
# workflow:rules controle SI le pipeline se lance
# C'est different de "rules:" dans un job (qui controle SI un job se lance)

workflow:
  rules:
    # Lancer le pipeline pour les merge requests
    - if: '$CI_PIPELINE_SOURCE == "merge_request_event"'

    # Lancer le pipeline pour la branche main
    - if: '$CI_COMMIT_BRANCH == "main"'

    # Lancer le pipeline pour les tags
    - if: '$CI_COMMIT_TAG'

    # Par default : NE PAS lancer le pipeline
    # (si aucune regle ne correspond, le pipeline est ignore)

  stages:
    - test
    - deploy

  test:
    stage: test
    script:
      - npm test

  deploy:
    stage: deploy
    script:
      - echo "Deploy"
    rules:
      - if: '$CI_COMMIT_BRANCH == "main"'
```

Comprendre la hierarchie des rules

```
workflow:rules  (le gardien a l'entree)
  |
  | Le pipeline se lance-t-il ? OUI/NON
  |
  v
job:rules  (le gardien de chaque salle)
  |
  | Ce job s'execute-t-il ? OUI/NON
  |
  v
Le job s'execute (ou pas)
```

IMPORTANT : Si workflow:rules bloque le pipeline, AUCUN job ne s'execute, meme si les rules du job correspondraient.

Les 3 types de conditions dans rules

GitLab propose 3 types de conditions pour les rules :

1. if - Condition sur les variables

```
# "if" evalue une expression avec des variables CI/CD
# C'est la condition la plus courante

job_example:
  script:
    - echo "Bonjour"
  rules:
    # Si c'est la branche main
    - if: '$CI_COMMIT_BRANCH == "main"'
      when: always

    # Si c'est un tag qui commence par "v"
    - if: '$CI_COMMIT_TAG =~ /^v\d+/'
      when: manual # Execution manuelle requise

    # Si le message de commit contient "[skip-tests]"
    - if: '$CI_COMMIT_MESSAGE =~ /\[skip-tests\]/'
      when: never # Ne jamais executer

    # Si c'est une merge request
    - if: '$CI_PIPELINE_SOURCE == "merge_request_event"'

    # Condition par default (si rien ne correspond)
    - when: never
```

Variables predefinies utiles pour les rules :

Variable	Description	Exemple
<code>\$CI_COMMIT_BRANCH</code>	Nom de la branche	main, develop, feature
<code>\$CI_COMMIT_TAG</code>	Nom du tag (vide si pas de tag)	v1.0.0
<code>\$CI_PIPELINE_SOURCE</code>	Source du pipeline	push, merge_request_event, web
<code>\$CI_MERGE_REQUEST_TARGET_BRANCH_NAME</code>	Branche cible de la MR	main
<code>\$CI_COMMIT_MESSAGE</code>	Message du dernier commit	fix: correction bug
<code>\$CI_COMMIT_TITLE</code>	Premiere ligne du message de commit	fix: correction bug

2. changes - Condition sur les fichiers modifies

```

# "changes" verifie si certains fichiers ont ete modifies
# Tres utile pour les monorepos !

test_frontend:
  stage: test
  script:
    - cd frontend && npm test
  rules:
    # Executer UNIQUEMENT si des fichiers dans frontend/ ont change
    - changes:
      - frontend/**/*
      - package.json
      - package-lock.json

test_backend:
  stage: test
  script:
    - cd backend && python -m pytest
  rules:
    # Executer UNIQUEMENT si des fichiers dans backend/ ont change
    - changes:
      - backend/**/*
      - requirements.txt

test_infra:
  stage: test
  script:
    - terraform plan
  rules:
    # Executer UNIQUEMENT si des fichiers Terraform ont change
    - changes:
      - terraform/**/*.tf
      - terraform/**/*.tfvars

```

Attention : changes fonctionne uniquement pour les pipelines de type push et merge_request_event. Pour les autres sources (web, schedule, api), la condition est TOUJOURS vraie (le job s'exécutera).

3. exists - *Condition sur l'existence de fichiers*

```
# "exists" verifie si un fichier existe dans le depot
# Utile pour des pipelines generiques reutilisables

test_node:
  stage: test
  image: node:18-alpine
  script:
    - npm ci
    - npm test
  rules:
    # Executer seulement si le projet a un package.json
    - exists:
        - package.json

test_python:
  stage: test
  image: python:3.11-alpine
  script:
    - pip install -r requirements.txt
    - pytest
  rules:
    # Executer seulement si le projet a un requirements.txt
    - exists:
        - requirements.txt

test_java:
  stage: test
  image: maven:3.9-eclipse-temurin-17
  script:
    - mvn test
  rules:
    # Executer seulement si le projet a un pom.xml
    - exists:
        - pom.xml
```

Combiner les conditions

On peut combiner if, changes et exists dans une meme regle.

Quand on les combine, c'est un **ET logique** (toutes les conditions doivent etre vraies).

```
deploy_frontend:
  stage: deploy
  script:
    - echo "Deploiement du frontend"
  rules:
    # Deployer seulement si :
    # - C'est la branche main (if)
    # - ET des fichiers frontend ont change (changes)
    - if: '$CI_COMMIT_BRANCH == "main"'
      changes:
        - frontend/**/*
      when: always

    # Deploiement manuel pour les merge requests
    - if: '$CI_PIPELINE_SOURCE == "merge_request_event"'
      changes:
        - frontend/**/*
      when: manual
      allow_failure: true # Le pipeline ne bloque pas si on ne clique pas

    # Par default : ne pas deployer
    - when: never
```

Merge request pipelines vs branch pipelines

C'est une source de confusion **tres courante** chez les debutants.

```
BRANCH PIPELINE (par default) :  
  Se declenche a chaque "git push" sur une branche  
  Variable : $CI_PIPELINE_SOURCE == "push"  
  Variable : $CI_COMMIT_BRANCH == "nom-de-la-branche"  
  
MERGE REQUEST PIPELINE :  
  Se declenche quand une Merge Request est creee ou mise a jour  
  Variable : $CI_PIPELINE_SOURCE == "merge_request_event"  
  Variable : $CI_MERGE_REQUEST_TARGET_BRANCH_NAME == "main"
```

Le probleme des pipelines en double :

Si vous ne faites pas attention, un git push sur une branche qui a une MR ouverte peut declencher **2 pipelines** : un branch pipeline ET un MR pipeline.

```
# SOLUTION : utiliser workflow:rules pour eviter les doublons  
  
workflow:  
  rules:  
    # Priorite 1 : MR pipelines (si une MR existe)  
    - if: '$CI_PIPELINE_SOURCE == "merge_request_event"'  
  
    # Priorite 2 : Branch pipeline pour main uniquement  
    - if: '$CI_COMMIT_BRANCH == "main"'  
  
    # Priorite 3 : Tag pipelines  
    - if: '$CI_COMMIT_TAG'  
  
    # Tout le reste est ignore (pas de double pipeline !)  
  
# Alternative : le template officiel de GitLab  
# Il gere automatiquement les doublons  
include:  
  - template: 'Workflows/MergeRequest-Pipelines.gitlab-ci.yml'
```

Definir des variables avec workflow:rules

```
# On peut definir des variables differentes selon le contexte
workflow:
  rules:
    - if: '$CI_COMMIT_BRANCH == "main"'
      variables:
        DEPLOY_ENV: 'production'
        LOG_LEVEL: 'warn'

    - if: '$CI_COMMIT_BRANCH == "develop"'
      variables:
        DEPLOY_ENV: 'staging'
        LOG_LEVEL: 'info'

    - if: '$CI_PIPELINE_SOURCE == "merge_request_event"'
      variables:
        DEPLOY_ENV: 'review'
        LOG_LEVEL: 'debug'

  stages:
    - test
    - deploy

  test:
    stage: test
    script:
      - echo "Tests avec LOG_LEVEL=$LOG_LEVEL"

  deploy:
    stage: deploy
    script:
      - echo "Deploiement sur $DEPLOY_ENV"
```

Partie 4 : Efficacite et optimisation des pipelines

L'analogie de la cuisine du restaurant

Imaginez une cuisine de restaurant ou les plats sont prepares dans un **ordre strict** :

1. Preparer l'entree
2. Preparer le plat principal
3. Preparer le dessert

Mais en realite, le dessert n'a **aucune dependance** avec l'entree !

Le patissier peut commencer le dessert en meme temps que le cuisinier prepare l'entree.

C'est exactement le probleme des pipelines lineaires : ils attendent inutilement.

Le **DAG** (Directed Acyclic Graph) permet de definir les vraies dependances.

Le probleme des pipelines lineaires

```

# Pipeline lineaire classique (LENT)
# Chaque stage attend que le precedent soit COMPLETEMENT termine

stages:
  - build
  - test
  - deploy

build_frontend:
  stage: build
  script:
    - npm run build:frontend # 5 minutes

build_backend:
  stage: build
  script:
    - npm run build:backend # 3 minutes

# Les tests attendent que TOUS les builds soient finis
# test_frontend attend build_backend alors qu'il n'en a pas besoin !
test_frontend:
  stage: test
  script:
    - npm run test:frontend # 4 minutes

test_backend:
  stage: test
  script:
    - npm run test:backend # 2 minutes

deploy:
  stage: deploy
  script:
    - echo "Deploy" # 1 minute

# Temps total : 5 + 4 + 1 = 10 minutes (en serie par stage)

```

DAG avec needs: - Le Game Changer

Le mot-cle **needs** : permet de definir les **dependances reelles** entre les jobs, independamment des stages.

```

# Pipeline avec DAG (RAPIDE)
# Chaque job démarre dès que ses dépendances sont terminées

stages:
  - build
  - test
  - deploy

build_frontend:
  stage: build
  script:
    - npm run build:frontend # 5 minutes
  artifacts:
    paths:
      - dist/frontend/

build_backend:
  stage: build
  script:
    - npm run build:backend # 3 minutes
  artifacts:
    paths:
      - dist/backend/

# test_frontend démarre dès que build_frontend est fini
# Il N'ATTEND PAS build_backend !
test_frontend:
  stage: test
  needs:
    - build_frontend # Dépendance explicite
  script:
    - npm run test:frontend # 4 minutes

# test_backend démarre dès que build_backend est fini
# Il N'ATTEND PAS build_frontend !
test_backend:
  stage: test
  needs:
    - build_backend # Dépendance explicite
  script:
    - npm run test:backend # 2 minutes

deploy:
  stage: deploy
  needs:
    - test_frontend
    - test_backend # Attend les 2 tests
  script:
    - echo "Deploy" # 1 minute

# Temps total : max(5+4, 3+2) + 1 = 9 + 1 = 10 minutes
# MAIS en pratique : build_backend (3min) finit avant build_frontend (5min)
# Donc test_backend démarre à 3min et finit à 5min
# Pendant ce temps, build_frontend finit à 5min et test_frontend démarre
# test_frontend finit à 9min, deploy à 10min
# Gain : le backend est testé en parallèle du build frontend !

```

Visualisation du DAG

```

SANS needs: (lineaire par stage)
=====
Temps:  0    1    2    3    4    5    6    7    8    9    10

Build:  [=== build_frontend ===][=== build_backend ==]
Test:                                     [== test_fe ==][test_be]
Deploy:                                     [deploy]

AVEC needs: (DAG - execution parallele)
=====
Temps:  0    1    2    3    4    5    6    7    8    9    10

          [===== build_frontend (5min) =====]
          [== build_backend (3min) ==]
                        [= test_backend =]
                                [==== test_frontend (4min) ====]
                                                [deploy]

```

needs: sans artifacts

Par default, needs: telecharge aussi les artifacts du job reference.

Si vous n'avez pas besoin des artifacts, vous pouvez le desactiver :

```

test_smoke:
  stage: test
  needs:
    - job: build_app
      artifacts: true      # Telecharger les artifacts (par default)

test_lint:
  stage: test
  needs:
    - job: build_app
      artifacts: false    # NE PAS telecharger les artifacts (plus rapide)
  script:
    - npm run lint        # Le lint n'a pas besoin du build

```

needs: vide pour demarrer immediatement

```

# Un job avec "needs: []" demarre IMMEDIATEMENT
# sans attendre aucun autre job, meme ceux des stages precedents

lint:
  stage: test
  needs: []                # Demarre tout de suite !
  script:
    - npm run lint

security_scan:
  stage: test
  needs: []                # Demarre tout de suite aussi !
  script:
    - npm audit

```

interruptible: - Annuler les pipelines obsoletes

Quand vous faites 3 git push en 5 minutes, vous avez **3 pipelines** en cours.

Les 2 premiers sont obsoletes puisque le 3eme contient les dernieres modifications.

interruptible: true dit a GitLab : "Tu peux annuler ce job si un nouveau pipeline demarre sur la meme branche."

```
# Les jobs de test sont interruptibles
# Si un nouveau push arrive, on les annule pour economiser des ressources

test_unitaires:
  stage: test
  interruptible: true    # Ce job peut etre annule
  script:
    - npm ci
    - npm test

lint:
  stage: test
  interruptible: true    # Ce job aussi
  script:
    - npm run lint

# ATTENTION : le deploiement ne doit PAS etre interruptible !
# Annuler un deploiement en cours peut laisser votre app dans un etat instable

deploy_production:
  stage: deploy
  interruptible: false    # NE JAMAIS annuler ce job (valeur par default)
  script:
    - kubectl apply -f k8s/
```

Configuration globale avec auto_cancel :

```
# Depuis GitLab 16.x, on peut configurer l'auto-annulation
workflow:
  auto_cancel:
    on_new_commit: interruptible  # Annuler les jobs interruptibles
    on_job_failure: none          # Ne rien faire si un job echoue
```

resource_group: - Limiter les deploiements simultanes

Imaginez que 2 developpeurs pushent en meme temps sur main.

Sans precaution, 2 deploiements se lancent **en parallele**, ce qui peut casser votre application.

resource_group: cree une **file d'attente** : un seul job a la fois peut s'executer pour un meme resource_group.

```
# Un seul deploiement a la fois sur chaque environnement

deploy_staging:
  stage: deploy
  resource_group: staging    # Un seul deploy staging a la fois
  script:
    - echo "Deploiement sur staging"
    - kubectl apply -f k8s/staging/

deploy_production:
  stage: deploy
  resource_group: production # Un seul deploy production a la fois
  script:
    - echo "Deploiement en production"
    - kubectl apply -f k8s/production/
  rules:
    - if: '$CI_COMMIT_BRANCH == "main"'
      when: manual
```

```
SANS resource_group :
Pipeline 1 : [==== deploy =====]
Pipeline 2 :      [==== deploy =====]    <- CONFLIT !

AVEC resource_group :
Pipeline 1 : [==== deploy =====]
Pipeline 2 :      [==== deploy =====]    <- Attend son tour
```

retry: - Gerer les echecs temporaires

Certains echecs sont **temporaires** : un registre Docker lent, un reseau instable, un serveur surcharge. Plutot que de relancer manuellement, retry: le fait pour vous.

```
# Retry simple : reessayer 2 fois maximum
test:
  stage: test
  retry: 2                # Si le job echoue, reessayer jusqu'a 2 fois
  script:
    - npm ci
    - npm test

# Retry avance : reessayer uniquement pour certains types d'erreurs
build_image:
  stage: build
  retry:
    max: 2                # Maximum 2 tentatives
    when:                  # Reessayer UNIQUEMENT pour ces erreurs :
      - runner_system_failure # Le Runner a plante
      - stuck_or_timeout_failure # Le job est bloque
      - scheduler_failure      # Probleme de planification
      - data_integrity_failure # Probleme de donnees
  script:
    - docker build -t mon-app .
    - docker push mon-app

# NE PAS reessayer pour les erreurs de script (bugs dans votre code)
# Un bug ne se corrige pas en reessayant !
```

Types d'erreurs disponibles pour retry:when :

Valeur	Description
--------	-------------

always	Toujours reessayer (par default)
unknown_failure	Erreur inconnue
script_failure	Echec du script (exit code != 0)
api_failure	Erreur de l'API GitLab
stuck_or_timeout_failure	Job bloque ou timeout depasse
runner_system_failure	Probleme du Runner
runner_unsupported	Runner incompatible
stale_schedule	Pipeline programme obsolete
job_execution_timeout	Timeout du job
archived_failure	Projet archive
scheduler_failure	Erreur du planificateur
data_integrity_failure	Probleme d'integrite des donnees

timeout: - Limiter la duree d'un job

Un job qui tourne indefiniment bloque votre Runner et gaspille des ressources.

`timeout:` definit une duree maximale apres laquelle le job est **tue**.

```
# Timeout global du projet : Settings > CI/CD > General pipelines > Timeout
# Par default : 1 heure

# Timeout par job (prioritaire sur le timeout global)
test_rapide:
  stage: test
  timeout: 5 minutes      # Ce job ne doit pas depasser 5 minutes
  script:
    - npm run test:unit

test_integration:
  stage: test
  timeout: 30 minutes     # Les tests d'integration sont plus longs
  script:
    - npm run test:integration

build_image:
  stage: build
  timeout: 15 minutes     # Le build Docker ne doit pas trainer
  script:
    - docker build -t mon-app .

deploy:
  stage: deploy
  timeout: 10 minutes     # Le deploiement doit etre rapide
  script:
    - kubectl apply -f k8s/
```

Combiner toutes les optimisations

Voici un exemple complet qui combine **toutes** les techniques vues dans cette partie :

```

# Pipeline optimise - Combinaison de toutes les techniques

workflow:
  auto_cancel:
    on_new_commit: interruptible
  rules:
    - if: '$CI_PIPELINE_SOURCE == "merge_request_event"'
    - if: '$CI_COMMIT_BRANCH == "main"'
    - if: '$CI_COMMIT_TAG'

stages:
  - prepare
  - build
  - test
  - deploy

# ===== PREPARE =====

install_deps:
  stage: prepare
  interruptible: true
  timeout: 5 minutes
  image: node:18-alpine
  script:
    - npm ci
  cache:
    key: ${CI_COMMIT_REF_SLUG}
    paths:
      - node_modules/
  artifacts:
    paths:
      - node_modules/
    expire_in: 1 hour

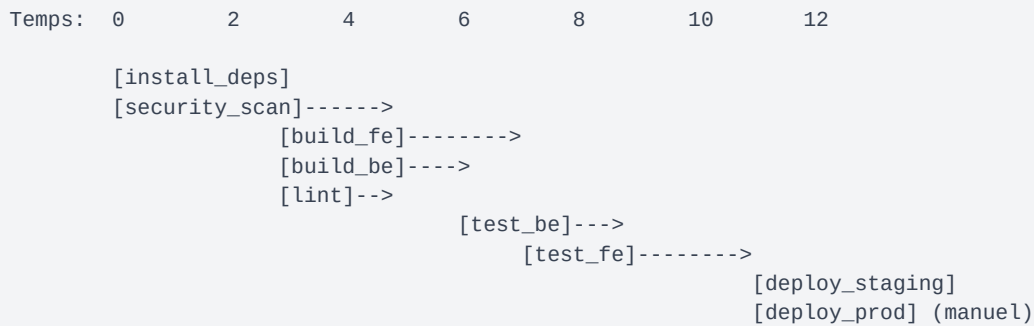
# ===== BUILD =====

build_frontend:
  stage: build
  interruptible: true
  timeout: 10 minutes
  needs:
    - install_deps
  script:
    - npm run build:frontend
  artifacts:
    paths:
      - dist/frontend/
    expire_in: 1 hour

build_backend:
  stage: build
  interruptible: true
  timeout: 10 minutes
  needs:
    - install_deps
  script:
    - npm run build:backend
  artifacts:

```

Schema recapitulatif du pipeline optimise



Comparaison des temps :

Pipeline lineaire : $\text{prepare}(2) + \text{build}(5) + \text{test}(5) + \text{deploy}(2) = 14 \text{ minutes}$
Pipeline optimise : $\max(\text{prepare} + \text{build_fe} + \text{test_fe}, \text{security}) + \text{deploy} = \sim 12 \text{ min}$
Gain : environ 15-20% plus rapide (et plus sur les projets complexes !)

Resume du Jour 3

Ce que nous avons appris aujourd'hui :

1. INCLUDES : Reutiliser et centraliser la configuration
 - include:local (meme projet)
 - include:file (autre projet GitLab)
 - include:template (templates officiels)
 - include:remote (URL externe)
 - trigger: pour declencher des pipelines cross-projet
 - Passage de variables entre pipelines
2. CHILD PIPELINES : Pipelines dynamiques
 - Pattern generate + trigger
 - Generation de YAML a la volee (shell, Python)
 - Detection de changements dans un monorepo
 - rules avec variables dans les child pipelines
3. WORKFLOW RULES : Contrôler le pipeline globalement
 - workflow:rules (gardien a l'entree)
 - if / changes / exists (3 types de conditions)
 - Merge request pipelines vs branch pipelines
 - Eviter les pipelines en double
 - Variables contextuelles avec workflow:rules
4. OPTIMISATION : Rendre les pipelines plus rapides et robustes
 - DAG avec needs: (dependances reelles)
 - interruptible: true (annuler les pipelines obsoletes)
 - resource_group: (un seul deploiement a la fois)
 - retry: (gerer les echecs temporaires)
 - timeout: (limiter la duree des jobs)

Demain (Jour 4) : Environnements, deploiement et strategies avancees !

Document cree pour la formation CI/CD - Jour 3

Niveau : Debutant a intermediaire

Duree estimee de lecture : 2 heures